

Лабораторная работа № 4

ИЗБЫТОЧНОЕ КОДИРОВАНИЕ ДАННЫХ В ИНФОРМАЦИОННЫХ СИСТЕМАХ. КОД ХЕММИНГА

Цель: приобретение практических навыков кодирования/декодирования двоичных данных при использовании кода Хемминга.

Задачи:

1. Закрепить теоретические знания по использованию методов помехоустойчивого кодирования для повышения надежности передачи и хранения в памяти компьютера двоичных данных.

2. Разработать приложение для кодирования/декодирования двоичной информации кодом Хемминга с минимальным кодовым расстоянием 3 или 4.

3. Результаты выполнения лабораторной работы оформить в виде описания разработанного приложения, методики выполнения экспериментов с использованием приложения и результатов эксперимента.

4.1. Теоретические сведения

4.1.1. Основные понятия и определения

Надежность системы – характеристика способности программного, аппаратного, аппаратно-программного средства выполнить при определенных условиях требуемые функции в течение конкретного периода времени.

Достоверность работы системы (устройства) – свойство, характеризующее истинность конечного (выходного) результата работы (выполнения программы), определяемое способностью средств контроля фиксировать правильность или ошибочность работы.

Ошибка устройства – неправильное значение сигнала (бита – в цифровом устройстве) на внешних выходах устройства или отдельного его узла, вызванное технической неисправностью, или воздействующими на него помехами (преднамеренными либо непреднамеренными), или иным способом.

Ошибка программы – проявляется в не соответствующем реальному (требуемому) промежуточном или конечном значении

(результате) вследствие неправильно запрограммированного алгоритма или неправильно составленной программы.

Как следует из вышеприведенного определения, надежность есть внутреннее свойство объекта, заложенное в него при изготовлении и проявляющееся во время эксплуатации. Вторая особенность надежности состоит в том, что она проявляется во времени. И третья особенность надежности выражается по-разному при различных условиях эксплуатации и различных режимах применения объекта (информационной системы в целом, отдельного ее блока, канала передачи сообщения, оперативной или внешней памяти компьютера).

 **Надежность является комплексным свойством, включающим в себя единичные свойства: безотказность, ремонтопригодность, сохраняемость, долговечность.**

Безотказность – это свойство технического объекта непрерывно сохранять работоспособное состояние в течение некоторого времени (или *наработки*). Наработка, как правило, измеряется в единицах времени.

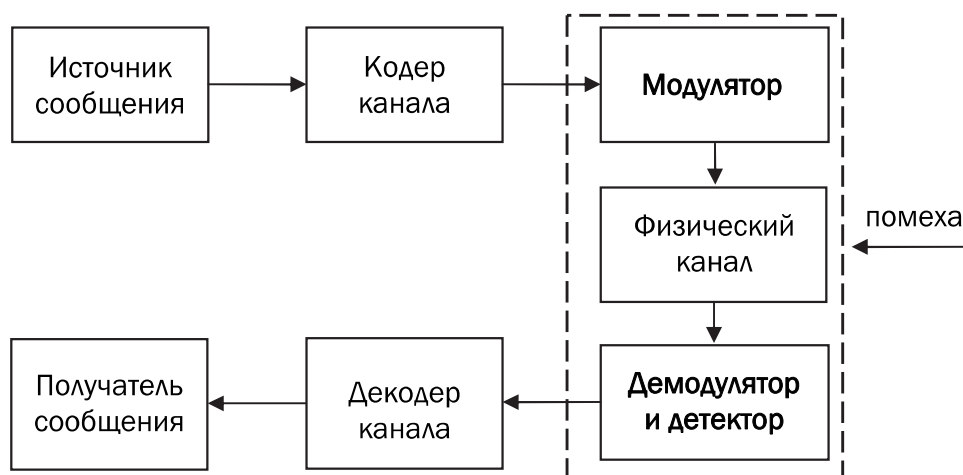
Ремонтпригодность – это свойство технического объекта, заключающееся в приспособленности к поддержанию и восстановлению работоспособного состояния путем технического обслуживания, ремонта (или с помощью дополнительных, избыточных технических средств, функционирующих параллельно с объектом). Большинство современных цифровых систем и устройств (в том числе компьютеры и компьютерные системы, отдельные блоки и модули компьютеров – полупроводниковая, магнитная или оптическая память) содержат специальные средства, призванные автоматически восстанавливать работоспособность этих объектов при нарушении нормального функционирования.

Такие специальные средства контроля называются *избыточными*. На рисунке приведена упрощенная структурная схема системы передачи данных с избыточными средствами аппаратного контроля.

Не вдаваясь в детали функционирования блоков, названия которых на схеме выделены полужирным, рассмотрим основные принципы работы представленной схемы в соответствии с задачами данной лабораторной работы. Как видим, сначала осуществляется формирование данных в виде двоичных символов. Затем *кодер* канала вносит в принятую информационную последовательность некоторую *избыточность* (данный процесс называется

кодированием или помехоустойчивым кодированием), которую декодер может использовать для исправления возникающих при передаче данных по каналу связи ошибок.

Таким образом, простейшая структурная схема дополнена двумя интересующими нас блоками: кодером (канала), осуществляющим преобразование исходного сообщения (информационного слова) X_k (длина сообщения – k символов) в избыточное сообщение (кодированное слово) X_n длиной n символов ($n > k$), и декодером (канала).



Структурная схема системы передачи данных с избыточными средствами аппаратного контроля

Изначальной причиной нарушения нормальной работы цифрового устройства являются технические *дефекты* (неисправности), возникающие внутри узлов или блоков устройства либо в каналах связи между ними.

Дефекты или неисправности могут приводить либо к кратковременному нарушению достоверности работы устройства (*сбой*), либо к полной и окончательной потере достоверности (*отказ*).

В каждом из этих случаев следствием неисправности являются ошибки в информации (*информационные ошибки*). Чаще всего причиной ошибок бывают внешние помехи, как это показано на рисунке. Количество таких ошибок (количество ошибочных двоичных символов) принято называть *кратностью ошибки*.

Пример. Исходная (правильная) информационная последовательность $X_k = 1001$. Длина этой последовательности равна 4 битам ($k = 4$). Некоторая из перечисленных причин привела к тому, что в этой последовательности появились две ошибки (*кратность*

ошибки равна двум): $Y_k = 1\bar{1}\bar{1}1$. Здесь Y_k – сообщение на выходе канала (без учета его кодирования/декодирования).

Обнаружение и/или исправление подобных ошибок как раз и призваны обеспечить кодер и декодер. Дополнительную информацию по рассматриваемым аспектам можно найти в пособиях [5, 9].

При использовании избыточных кодов исходные данные делятся на блоки из k битов (называются информационными битами). В процессе кодирования каждый k -битный блок данных преобразуется, как было отмечено выше, в блок из n битов (кодовое слово). Число k часто называется *размерностью кода*. Таким образом, к каждому блоку данных в процессе кодирования присоединяются $r = n - k$ битов, которые называют *избыточными битами* (redundant bits), *битами четности* (parity bits) или *контрольными битами* (check bits); новой информации они не несут.

Для обозначения описанного кода обычно пользуются записью (n, k) и говорят, что данный код использует n символов для передачи (хранения) k символов сообщения. Отношение числа битов данных к общему числу битов k/n именуется степенью кодирования (code rate) – доля кода, которая приходится на полезную информацию. Еще одним важным параметром кода является *расстояние Хемминга* (d), которое показывает, что два кодовых слова различаются по крайней мере в d позициях.

В общем случае код позволяет обнаруживать t_o ошибок:

$$t_o = \begin{cases} \frac{d}{2}, & d - \text{четное}; \\ \frac{d-1}{2}, & d - \text{нечетное}. \end{cases}$$

Количество исправляемых кодом ошибок t_n определяется следующим образом:

$$t_n = \begin{cases} \frac{d-1}{2}, & d - \text{нечетное}; \\ \frac{d-2}{2}, & d - \text{четное}. \end{cases}$$

Во всех этих простых математических выражениях d соответствует минимальному кодовому расстоянию Хемминга анализируемого кода.

4.1.2. Теоретические основы линейных блочных кодов

К. Шеннон сформулировал теорему для случая передачи дискретной информации по каналу связи с помехами, утверждающую, что вероятность ошибочного декодирования принимаемых сигналов может быть обеспечена сколь угодно малой путем выбора соответствующего способа кодирования сигналов. В теореме Шеннона не говорится о том, как нужно строить необходимые помехоустойчивые коды. Однако в ней указывается на принципиальную возможность кодирования, при котором может быть обеспечена сколь угодно высокая надежность передачи.

Код Хемминга относится к классу линейных блочных кодов.

Линейные блочные коды – это класс кодов с контролем четности, которые можно описать парой чисел (n, k) .

Для формирования r проверочных символов (кодирования), т. е. вычисления проверочного слова X_r , используется *порождающая матрица* G : совокупность базисных векторов будем далее записывать в виде матрицы G размерностью $k \times n$ с единичной подматрицей (I) в первых k строках и столбцах:

$$G = [P | I]. \quad (4.1)$$

Более точно матрица G называется *порождающей матрицей линейного корректирующего кода в приведенно-ступенчатой форме*. Кодовые слова являются линейными комбинациями строк матрицы G (кроме слова, состоящего из нулевых символов).

Кодирование заключается в умножении вектора сообщения X_k длиной k на порождающую матрицу по правилам матричного умножения (все операции выполняются по модулю 2). Очевидно, что при этом первые k символов кодового слова равны соответствующим символам сообщения, а последние r символов образуются как линейные комбинации первых [5, 9].

Для всякой порождающей матрицы G существует матрица H размерности $r \times n$, задающая базис нулевого пространства кода и удовлетворяющая равенству

$$G \cdot H^T = 0. \quad (4.2)$$

Справедливо также

$$X_n \cdot H^T = H \cdot (X_n)^T = 0. \quad (4.3)$$

В последнем выражении символ « T » означает *транспонирование*, а $X_n = x_1, x_2, \dots, x_n$.

Матрица H , называемая *проверочной*, равна

$$H = \left[-P^T \mid I \right] = P^* \mid I. \quad (4.4)$$

В коде Хемминга с минимальным кодовым расстоянием $d_{\min} = 3$ проверочная матрица H имеет классический вид и состоит из двух подматриц: P^* размером $k \times r$ и I размером $r \times r$ соответственно.

В последнем выражении I – единичная матрица порядка r ($r \times r$).

Количество r избыточных (проверочных) символов кодового слова определяется из следующей простой логической цепи рассуждений.

Общее число всех возможных комбинаций 2^r должно удовлетворять неравенству

$$2^r \geq n + 1$$

в силу того, что

$$\begin{aligned} 2^r \geq n + 1 &\Rightarrow 2^r \geq k + r + 1 \Rightarrow r \geq \log_2(k + r + 1) \Rightarrow \\ &r \geq \log_2(k + 1). \end{aligned}$$

Присутствие цифры «1» в приведенных выражениях соотносит ее с нулевым вектор-столбцом, который в матрице не используется.

Например, для (7, 4)-кода Хемминга проверочная матрица в упорядоченном виде выглядит так:

$$H_{(7,4)} = \begin{vmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{vmatrix}.$$

Результат умножения сообщения на выходе канала передачи (Y_n) или (что равнозначно) сообщения, считываемого из памяти, на проверочную матрицу (H) называется **синдромом (вектором ошибки) S** :

$$S = H \cdot (Y_n)^T = Y_n \cdot H^T, \quad (4.6)$$

где $Y_n = y_1, y_2, \dots, y_n$ – принятый вектор (сообщение на выходе канала), полученный после передачи либо считывания из памяти. Вектор Y_n обычно представляют в следующем виде:

$$Y_n = X_n + E_n, \quad (4.7)$$

где $E_n = e_1, e_2, \dots, e_n$ – вектор ошибки.

Синдром – это результат проверки четности, выполняемой над сообщением Y_n для определения его принадлежности заданному набору кодовых слов. При положительном результате проверки синдром S равен 0, т. е. $Y_n = X_n$. Если Y_n содержит ошибки, которые можно исправить, то синдром имеет определенное ненулевое значение, что позволяет обнаружить и исправить конкретную ошибочную комбинацию.

Важно запомнить, что в силу выражений (4.3)–(4.7) ненулевой синдром всегда равен сумме по модулю 2 тех вектор-столбцов матрицы H , номера которых соответствуют номерам ошибочных битов в слове Y_n .

Корректирующая способность кода Хемминга с $d_{\min} = 3$ может быть увеличена введением дополнительной проверки на четность. В этом случае проверочная матрица будет иметь вид:

$$H' = \left[\begin{array}{cccc|c} & & & & 0 \\ & & & & 0 \\ & & & & \vdots \\ & & & & 0 \\ \hline 1 & 1 & \dots & 1 & 1 \end{array} \right] \quad (4.8)$$

Так, минимальное кодовое расстояние такого кода будет равно 4: $d_{\min} = 4$. Такой код может исправлять все единичные ошибки с одновременным обнаружением всех двойных в анализируемом кодовом слове.

При этом нужно помнить, что вид матрицы (4.8) не соответствует ее каноническому представлению, поскольку во всех столбцах единичной матрицы, кроме последнего, будет по 2 единицы.

Для придания матрице канонического вида необходимо сложить посимвольно все строки между собой и результат сложения записать в последнюю строку (под горизонтальной линией в (4.8)).

4.2. Практическое задание

1. На основе информационного сообщения, представленного символами русского/английского алфавитов, служебными символами и цифрами, содержащегося в некотором текстовом файле (согла-

совать с преподавателем), сформировать информационное сообщение в двоичном виде; длина сообщения в бинарном виде должна быть не менее 16 символов. Для выполнения этого задания можно использовать коды ASCII символов алфавита либо результаты лабораторной работы № 3.

2. Для полученного информационного слова построить проверочную матрицу Хемминга (значение минимального кодового расстояния согласовать с преподавателем).

3. Используя построенную матрицу, вычислить избыточные символы (слово X_r).

4. Принять исходное слово со следующим числом ошибок: 0, 1, 2. Позиция ошибки определяется (генерируется) случайным образом.

5. Для полученного слова $Y_n = Y_k, Y_r$, используя уже известную проверочную матрицу Хемминга, вновь вычислить избыточные символы (обозначим их Y_r'), используя выражение (4.6).

6. Вычислить и проанализировать синдром. В случае, если анализ синдрома показал, что информационное сообщение было передано с ошибкой (или 2 ошибками), сгенерировать унарный вектор ошибки $E_n = e_1, e_2, \dots, e_n$ и исправить одиночную ошибку, используя формулу (4.7); проанализировать ситуацию при возникновении ошибки в 2 битах.

7. Результаты оформить в виде отчета по установленным правилам.

Файл анализируемой информации (а соответственно и интерфейс приложения) должен содержать исходное информационное сообщение, значения величин k, r, n , проверочную матрицу Хемминга $H_{n,k}$, слово X_n, X_r, Y_n, Y_r, Y_r' , синдром S , вектор ошибки E_n . Программа не должна быть чувствительна к длине информационного сообщения.

Некоторой сложностью, с которой можно столкнуться при выполнении данной лабораторной работы, является выбор и реализация алгоритма построения проверочной матрицы Хемминга.

Существует несколько способов, позволяющих заполнить матрицу Хемминга. Одним из простых является следующий.

Поскольку известно, что минимальный вес столбцов в подматрице P' (см. выражение (4.5)) равен 2, вычисляем значение биннома Ньютона по следующей формуле:

$$C_{wt}^r = \frac{r!}{wt(r-wt)!},$$

где $wt = 2$, а $r = \log_2 k + 1$; wt – это вес (количество единиц).

Бином Ньютона позволяет вычислить количество различных комбинаций из нулей и единиц, которые мы можем получить при заданных значениях r и wt . Ниже приведен синтаксис функций, позволяющих вычислить бином Ньютона (листинг 4.1).

```

:: int fact (int n) //функция вычисления
::             //факториала натурального числа n
:: {
:: int answer; //переменная, которая будет хранить
::             //значение факториала натурального числа n
:: if (n==1)
:: return 1;   //1 (1!=1)
::             //рекурсивное обращение функции fact() к самой себе
:: answer=fact(n - 1)*n;
:: return (answer); //возвращение результата
::             //работы функции в место
::             //вызова ее в теле основной программы
:: }
::
::             //функция вычисления бинома Ньютона
:: int binom(int wt,int r) {
:: int C;      //переменная, которая будет хранить значение
::             //бинома Ньютона
:: C=fact(r)/(fact(wt)*fact(r - wt));
:: return (C); //возвращение результата работы
::             //функции в место вызова ее в теле основной программы
:: }

```

Листинг 4.1. Пример реализации алгоритма для вычисления бинома Ньютона (числа сочетаний)

Далее переводя натуральные числа, начиная с 3 (первое натуральное число, которое при переводе в двоичную систему счисления имеет вес, равный 2) в двоичную систему счисления и дополняя полученную комбинацию нулями до r позиций (если это необходимо), по очереди заполняем столбцы подматрицы P' .

Например, положим, что $r = 5$, $wt = 2$.

Тогда

$$C_2^5 = \frac{5!}{2(5-2)!} = 10.$$

$3_2 = 11$, т. е. первым столбцом матрицы P' будет вектор $\underbrace{[00\ 0\ 11]}$

дополнено в соответствии со значением r

$4_2 = 100 - wt(4_2) = 1$ – не подходит,

$5_2 = 101 - wt(5_2) = 2$.

Следовательно, второй столбец матрицы $A_{k,r}$: 00101, и т. д.

Программно это может выглядеть в соответствии с листингом 4.2.

```

:: itoa(num,buf,2); //представление натурального
::                      //числа в двоичной системе счисления
:: int len=strlen(buf);
::                      //переменная len хранит длину
::                      //полученной двоичной последовательности
:: sum=0; //переменная, используемая для вычисления
::                      //веса полученной двоичной последовательности
::                      //(двоичного представления натурального числа n)
:: for (int i=0;i<len;i++)
:: {
::     if (buf[i]=='1')
:: sum++; //при встрече в двоичном
::         //представлении числа символа '1' счетчик sum
::         //увеличивается на 1
:: }
:: if (sum==wt) //если вес числа в двоичном
::             //представлении равен текущему
::             //весу (первоначально wt=2)
:: {
::     count++; //тогда число комбинаций,
::             //подходящих для заполнения
::             //столбцов матрицы Ak,r,
::             //увеличивается на 1
::     if (len<r) //если длина полученной
::             //двоичной последовательности
::             //меньше r, происходит
::             //увеличение ее на (r - длина
::             //последовательности) нулей
::     {
::         for (int j=0;j<(r - len);j++)
::         {
::             char buf1[80]='0';

```

```

:: strcat(buf1,buf);
:: strcpy(buf,buf1);
:: }
:: }
:: for (int i=0;i<r;i++){
::     //заполнение столбца матрицы Ak,r
::     A[i][count - 1]= buf[i];
:: }
:: num++;    //переход к следующему натуральному числу

```

Листинг 4.2. Программная реализация алгоритма создания проверочной матрицы кода Хемминга с $d_{\min} = 3$

В случае если все комбинации при заданных r и wt уже исчерпаны, увеличиваем вес wt столбцов матрицы на единицу и опять вычисляем бином Ньютона.

В заполнении матрицы I нет никаких сложностей. Для начала элементы матрицы инициализируются нулями. Если общее количество столбцов матрицы $H_{n,r} n = k + r$, а первый столбец матрицы I имеет индекс k (нумерация элементов матрицы $H_{n,r}$ начинается с 0), тогда это можно реализовать с помощью кода, представленного на листинге 4.3.

```

:: for (int i=k; i<n; i++)
:: {
::     for (int j=0;j<r;j++)
::     {
::         if (j=i - k) I[i,j]=1;
::         else I[i,j]=0;
::     }
:: }

```

Листинг 4.3. Фрагмент кода для заполнения столбцами единичной матрицы

Далее при вычислении i -го символа слова X_r достаточно подсчитать количество пар (1; 1) в соответствующих позициях слова X_k и i -й строки подматрицы P' . Если количество таких пар четное, т. е. по модулю 2 равно 0, то i -й элемент слова $X_r[i]$ равен 0, в обратном случае – 1.

Вычисление синдрома – это сложение по модулю 2 векторов (слов) Y_r и Y_r' . Пусть размерность этих векторов равна r (нумера-

цию элементов векторов примем с 0). Тогда можно использовать код, показанный на листинге 4.4.

```
:: for (int i=0;i<r;i++)
:: {
::   if (Yr[i]<>Yr'[i])
::     s[i]=1;
::   else s[i]=0;
:: }
```

Листинг 4.4. Фрагмент кода для вычисления синдрома

Из последнего следует, что i -й элемент синдрома S будет равен 0, если i -е элементы векторов Y_r и Y_r' равны между собой, и 1 – в противном случае.

Если анализ синдрома показал, что переданное слово содержит ошибку, ищем номер позиции в слове Y_n , в которой эта ошибка возникла. Зная позицию ошибки, можем сформировать унарный вектор ошибки E_n .

Пусть позицию ошибки хранит некоторая переменная *mist*. Первоначально необходимо проинициализировать все элементы вектора E_n нулями, далее выполнить анализ и вычисления в соответствии с листингом 4.5.

```
:: for (int i=0;i<n;i++)
:: {
::   if (i<>mist) E[i]=0;
::   else E[i]=1;
:: }
```

Листинг 4.5. Фрагмент кода программы для определения вектора ошибки

Чтобы исправить ошибку, необходимо сложить по модулю 2 векторы Y_n и E_n в соответствии с выражением (4.7).

ВОПРОСЫ ДЛЯ КОНТРОЛЯ И САМОКОНТРОЛЯ

1. В чем заключается цель и функциональная сущность преобразования информации на основе избыточного кодирования?

2. Пояснить зависимость r от длины информационного слова k . Охарактеризовать относительную избыточность сообщения и время его передачи по сети.

3. Записать проверочную матрицу кода Хемминга с $d_{\min} = 3$ и $d_{\min} = 4$ для $k = 4; 6; 8; 9; 10; 15; 16$.

4. Записать проверочную матрицу кода простой четности для k из вопроса 3. Пояснить на примере определение минимального кодового расстояния Хемминга для данного кода.

5. Предположим, есть выбор (при построении матрицы кода) между вектор-столбцами большего и меньшего веса. Какой вариант Вы предпочтете и почему?